

DAIFOOTCARE

On-Device Deep Learning for Diabetic-Foot-Ulcer Assessment

Model Design, Training, and Evaluation

Segmentation · Tissue Classification · Infection & Ischaemia

A technical reference for the three-model analysis pipeline, prepared as supporting documentation for doctoral research.

Version 1.1 · Last updated 26 July 2026

Supersedes v1.0 (2 July 2026). See the *Revision note* overleaf.

Contents

Revision note (v1.1)	2
1 Overview and System Design	3
1.1 Design principle: one shared backbone	3
1.2 Model summary	4
1.3 Datasets used across the project	4
2 Model 1 — Wound Segmentation & Measurement	5
2.1 Purpose and task	5
2.2 Datasets	5
2.3 Architecture and loss	5
2.4 Measurement post-processing	6
2.5 Results	6
2.6 Exported artefacts	7
2.7 Limitations	7
3 Model 2 — Tissue Classification	8
4 Model 3 — Infection & Ischaemia	10
5 Shared Preprocessing	12
6 Application Integration	12
7 References	13

Revision note (v1.1)

WHAT CHANGED SINCE V1.0

This revision records a change to the mobile application's measurement output. The **estimated wound depth** has been removed from the pipeline and the user interface. A single two-dimensional photograph cannot recover true wound depth; the earlier value was a pixel-density heuristic rather than a genuine three-dimensional measurement, and the optional manual-probe entry and reference-object scale-calibration steps were retired to simplify the capture flow. Model 1 now reports **length, width, and area** only. The segmentation network, its training procedure, and all reported metrics are *unchanged*; only the derived-measurement post-processing and its documentation are affected. Sections 1.2, 2.1, 2.4, 2.5, 2.7 and 6, and Figure 1, have been updated accordingly.

1 Overview and System Design

This document is the technical reference for the machine-learning components of DaiFootCare, a mobile application that performs automated assessment of diabetic foot ulcers (DFU) entirely on-device. Three deep-learning models cooperate in a single inference pipeline: a wound-segmentation model that produces pixel-level masks and derived clinical measurements, a tissue-classification model that identifies the tissue types present in the wound bed, and an infection-and-ischæmia model that estimates two clinically-validated risk indicators. Each section below documents the purpose, data provenance, architecture, training procedure, evaluation protocol, results, and limitations of a model, at a level of detail intended to support reproduction and academic reporting.

1.1 Design principle: one shared backbone

The central architectural decision of the system is that Models 2 and 3 share a single frozen CLIP ViT-B/32 image encoder (`clip_backbone_fp16.tflite`). The backbone is the only large asset in the deployment. It is loaded once and executed once per input image to produce a 512-dimensional embedding; each downstream task then contributes only a small “head” network — 20 MB for tissue classification and 262 KB for infection/ischæmia. Model 1 (segmentation) is standalone. This factoring keeps the installed application small and keeps per-image inference fast, because the expensive encoder pass is amortised across two clinical tasks.

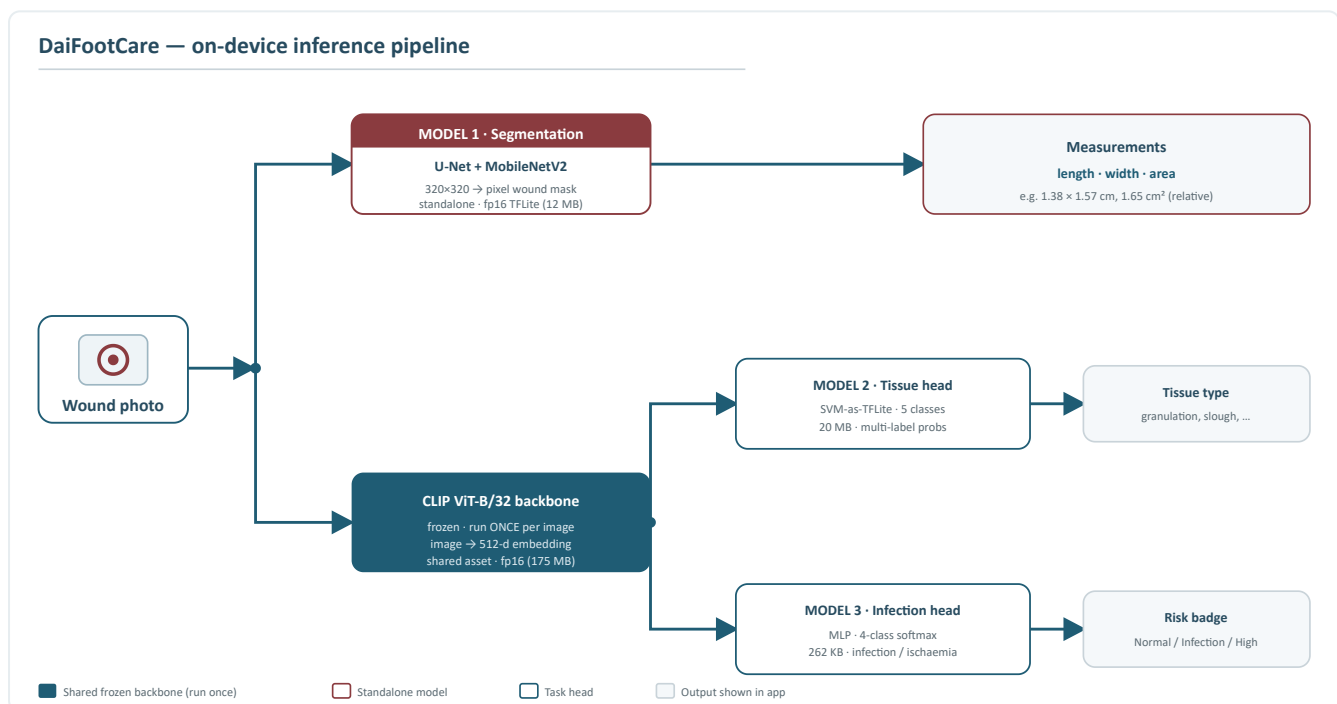


Figure 1. High-level dataflow. The CLIP backbone is evaluated a single time per image and feeds both classification heads; Model 1 runs independently to produce measurements. As of v1.1 the measurement output is length, width and area only — depth has been removed (see the Revision note).

1.2 Model summary

#	Model	Task	Backbone	Deployed file(s)	Headline result
1	Wound Segmentation & Measurement	pixel mask → length / width / area	U-Net + MobileNetV2	model1_wound_fp16.tflite (12 MB)	Test Dice 0.818 · val acc 0.89
2	Tissue Classification	5 tissue types present in wound	CLIP ViT-B/32 (frozen) + SVM head	clip_backbone_fp16.tflite (175 MB) + tissue_head.tflite (20 MB)	AUC 0.825 / F1 0.733
3	Infection & Ischaemia	infection + blood-flow status	CLIP ViT-B/32 (frozen, shared) + MLP head	infection_ischaemia_head.tflite (262 KB)	Infection AUC 0.890 · Ischaemia AUC 0.987

Table 1. The three models, their tasks, backbones, deployed artefacts, and headline evaluation metrics.

1.3 Datasets used across the project

Five source collections are used across the three models: DFUTissue and DFUTissueSegNet [6] (finely-annotated tissue and wound masks), DFUC2020 and DFUC2021 (the Diabetic Foot Ulcer Challenge collections [7]), and dfu-extra-labels, a set of doctor-reviewed annotations added specifically for this project. The role each dataset plays differs by model and is documented in the relevant section. For the origin, licensing, and inter-relationship of the Challenge datasets, readers are referred to the official DFU dataset overview [7]; the specific Challenge papers required by the dataset providers are cited in full in Section 7.

2 Model 1 — Wound Segmentation & Measurement

Notebook: `model-1-kag.ipynb` (Kaggle, GPU). Deployed artefact: `model1_wound_fp16.tflite`.

2.1 Purpose and task

Model 1 locates the wound region in a photograph as a pixel-level mask and then converts that mask into clinical measurements: **length, width, and area**. Formally the task is binary semantic segmentation — every pixel is classified as either wound or background. The network consumes an image tensor of shape `[1, 320, 320, 3]` (float32, values in `[0,1]`) and produces a per-pixel wound-probability mask of shape `[1, 320, 320, 1]`. A deterministic post-processing stage converts this mask into measurements.

ON WOUND DEPTH

Depth is deliberately **not** produced. A single 2-D photograph, taken with an ordinary phone camera and no reference geometry, does not contain the information needed to recover physical wound depth. An earlier build reported a depth figure derived from wound-pixel density; this was a heuristic, not a genuine three-dimensional measurement, and it was removed in v1.1. Depth, where clinically required, must be obtained from a sterile probe or a dedicated depth sensor and is out of scope for the on-device pipeline.

2.2 Datasets

Dataset	Role	Notes
DFUTissue (Labeled / Original)	Primary target — precise pixel masks	≈75 finely-annotated images; the true supervision signal. Validation and test use only these, so reported metrics reflect genuine wound-boundary quality.
DFUC2020	Weak pseudo-masks (bounding boxes)	Coarse box-derived masks used for training only, capped at ≈250 examples to avoid pulling the model away from precise boundaries.
DFUC2021	Unlabeled images	Used solely to supply the representative dataset for int8 export; it is a classification set with no localisation labels.

Table 2. Data sources for Model 1 and the distinct role each plays in training and evaluation.

To make the most of the small precise set, the finely-annotated images (from DFUTissue [6], with the weak box masks drawn from DFUC2020 [9,10]) are oversampled ×3 relative to the box masks, and strong augmentation (horizontal/vertical flips, rotations, and random cutout/erasing) is applied to improve generalisation and counter overfitting on the roughly 75 precise examples.

2.3 Architecture and loss

- **Backbone:** a U-Net decoder [1] built on a pretrained MobileNetV2 [2] encoder (ImageNet transfer learning). Encoder skip connections are taken at input `/2`, `/4`, `/8`, `/16` with the bottleneck at `/32`; the backbone input is internally rescaled to `[-1, 1]`. A `320×320` input (`÷32`) preserves finer wound-boundary detail than a `256×256` input.
- **Loss:** $0.5 \times \text{BCE} + \text{Focal-Tversky}$ [3]. The Tversky term uses $\alpha = 0.3$, $\beta = 0.7$, which penalises missed wound pixels more heavily to raise recall and Dice; the focal component ($\gamma = 0.75$) concentrates learning on the small wound region.

Training schedule (multi-phase)

1. **Phase 1** — freeze the encoder and train the decoder only (25 epochs, learning rate `1e-3`).
2. **Phase 2** — fine-tune the top 36 of 155 encoder layers with batch-normalisation frozen (learning rate `5e-5`).
3. **Self-training** — add high-confidence pseudo-labels and continue training; the best-of-both checkpoint is retained by validation loss.

Callbacks and regime: `ModelCheckpoint` (best val_loss), `ReduceLROnPlateau`, and `EarlyStopping` with best-weight restoration. Mixed-precision training is used on GPU.

2.4 Measurement post-processing

The mask-to-measurement conversion is deterministic and is ported one-to-one from the training notebook into the app's Dart code, guaranteeing that on-device measurements match the reference implementation:

1. Binarise the probability mask (default threshold 0.5, with a half-peak fallback if no pixel clears the threshold).
2. Apply a 5×5 morphological open then close to remove specks and fill small holes.
3. Retain the largest connected component as the wound.
4. **Length and width** are the extents along the wound's principal (PCA) axes — i.e. the minimum-area rectangle enclosing the component, equivalent to `cv2.minAreaRect` in the notebook rather than an axis-aligned bounding box — scaled to centimetres. **Area** is the wound-pixel count × scale². The scale is a pixels-per-cm factor.
5. **Scale (always uncalibrated in v1.1)**. The application no longer collects a reference-object calibration, so no true pixels-per-cm factor is available. The wider side of the frame is assumed to span `_assumedFrameCm = 12 cm`, the derived factor scales with image resolution, and the result is flagged `isCalibrated = false`. Consequently the centimetre values are approximate; the reliable healing signal is the **relative area trend over time**, not absolute centimetres.

REMOVED STEP

The former step 8, an estimated depth = wound-pixel density × 1.5 mm, has been deleted. It was a heuristic, not a true 3-D measurement, and no longer forms part of the pipeline.

2.5 Results

Two families of metrics are reported. First, per-epoch training metrics logged by Keras at the selected checkpoint (best validation loss, epoch 2 of the fine-tuning phase): pixel accuracy, intersection-over-union (IoU / Jaccard), and the differentiable soft-Dice coefficient. Second, the final segmentation quality computed after the full post-processing pipeline (binarisation, morphology, largest-component selection) on the held-out precise masks, which is the figure that reflects true wound-boundary quality.

Split	Pixel accuracy	IoU (Jaccard)	Dice
Training (selected checkpoint)	0.945	0.597	0.746
Validation (per-epoch, Keras)	0.892	0.526	0.689
Validation (final, post-processed)	—	—	0.706
Test (held-out, 8-view TTA)	—	—	0.818

Table 3. Segmentation performance. Training/validation pixel accuracy, IoU and soft-Dice are per-epoch Keras metrics at the selected checkpoint (epoch 2). The final post-processed validation Dice (0.706) and the held-out test Dice (0.818, 8-view test-time augmentation) are the primary quality figures. Best mask threshold on validation: 0.58.

READING THE METRICS

Pixel accuracy is high (≈0.89–0.95) partly because background pixels dominate the frame, so a model is “correct” on most pixels before locating the wound at all. For this reason Dice and IoU are the primary segmentation metrics and pixel accuracy is reported only as a supporting figure. The per-epoch (soft) Dice and the final post-processed Dice differ slightly (0.689 vs. 0.706 on validation) because the latter is computed after binarisation and morphological clean-up rather than on the raw probability map.

Example output:

```
{ length 1.38 cm, width 1.57 cm, area 1.65 cm2 } // approximate — isCalibrated = false
```

2.6 Exported artefacts

- `model1_wound_fp16.tflite` — 12.1 MB — deployed in the application.
- `model1_wound_int8.tflite` — 6.5 MB — a smaller alternative; int8 quantisation makes the derived measurements noisier.

2.7 Limitations

LIMITATIONS OF MODEL 1

Absolute centimetre values are not reliable without a calibration reference object in the frame. Because the application runs uncalibrated (it assumes a 12 cm frame width and flags `isCalibrated = false`), only the **relative area trend** over successive visits should be treated as a healing signal; single-visit centimetre figures are indicative only. Wound **depth is not measured** at all — it cannot be recovered from a 2-D image and is out of scope for this pipeline. Finally, the precise-mask training set is small (≈ 75 images), which augmentation and the weak DFUC2020 box masks only partly compensate for.

3 Model 2 — Tissue Classification

Notebook: `dfut-model13-newdata-gold.ipynb`. Deployed: `clip_backbone_fp16.tflite` + `tissue_head.tflite` (+ `tissue_head_meta.json`).

3.1 Purpose and task

Model 2 identifies which tissue types are visible in the wound bed. Because several tissue types can be present simultaneously, this is a multi-label classification problem rather than a single-label one. A CLIP-preprocessed wound image is mapped to a 512-dimensional embedding, from which five independent probabilities are produced. A tissue is judged present when its probability meets or exceeds its individually-tuned threshold.

Classes (5): epithelial, granulation, necrosis, callus, slough. The fibrin class was dropped because the clean evaluation set contained only two positive examples, too few to score reliably.

CLASSIFICATION, NOT SEGMENTATION

Model 2 answers “is each tissue type present in this wound?” at the image level. It does not produce pixel masks of tissue regions — that distinction matters when interpreting its outputs against the pixel-level Model 1.

3.2 Datasets

Dataset	Role
DFUTissue	Tissue-labeled wound images.
Source A doctor labels (<code>dfu-extra-labels</code>)	367 doctor-reviewed images (<code>...originalimagereNamed....xlsx</code>).
DFUC2020 doctor labels (<code>dfuc2020_annotatations-4.csv</code>)	810 training images carrying real 6-class doctor labels (previously only bounding boxes).

Table 4. Tissue-classification data sources. Doctor-reviewed labels were added specifically for this project.

Pooling these sources yields a gold set of 1,176 doctor-labeled images; training is further expanded by augmentation. Evaluation is reported on the 366 Source-A images so that results are directly comparable to the historic best baseline.

3.3 Architecture

- **Encoder:** a frozen CLIP ViT-B/32 [4] image encoder producing a 512-d embedding. This embedding carries the predictive power — a from-scratch CNN scored near-random (≈ 0.55 AUC).
- **Head:** a per-class RBF-SVM trained on the CLIP embeddings.
- **Mobile export:** the SVM is reproduced exactly as TensorFlow operations (“SVM-as-TFLite”), verified to within $8e-6$ of the desktop SVM, and exported as `tissue_head.tflite`.
- **Research vs. mobile:** the research-best desktop variant rank-averages CLIP $\oplus$ DINOv2 [5], but DINOv2 (ViT-L, ≈ 1.2 GB) is impractical on phones, so the deployed mobile build is CLIP-only.

3.4 Results (Source-A evaluation)

Variant	AUC	F1
Historic best	0.835	0.656
Desktop best (CLIP $\oplus$ DINOv2 ensemble)	0.838	0.726
Mobile (CLIP + SVM-as-TFLite, deployed)	0.825	0.733

Table 5. Tissue-classification results. The deployed mobile model exceeds the historic best on F1 by +0.077 while running fully on-device.

3.5 Output and thresholds

The model ships raw probabilities; the application applies per-class thresholds read from `tissue_head_meta.json`: epithelial 0.09, granulation 0.43, necrosis 0.60, callus 0.45, slough 0.63. The stored cross-validation metrics are AUC 0.825 and F1 0.7327.

3.6 Files

File	Size	Role
<code>clip_backbone_fp16.tflite</code>	175 MB	image (1,224,224,3) → 512-d embedding (shared with Model 3)
<code>tissue_head.tflite</code>	20 MB	512-d → 5 probabilities
<code>tissue_head_meta.json</code>	—	class order, thresholds, cross-validation metrics

Table 6. Deployed artefacts for Model 2.

3.7 Limitations

LIMITATIONS OF MODEL 2

The mobile build is CLIP-only (no DINOv2), trading a small accuracy margin for a large reduction in size. The fibrin class is not scoreable because too few labelled examples exist.

4 Model 3 — Infection & Ischaemia

Script: `train_model3_infection.py`. Deployed: shared CLIP backbone + `infection_ischaemia_head.tflite` (+ `infection_ischaemia_head_meta.json`). Full detail in `MODEL3_INFECTION_HANDOFF.md`.

4.1 Purpose

Model 3 replaces an originally-planned “Pus Level” output, for which no pus-severity labels exist. Instead it predicts two clinically-validated indicators: Infection and Ischaemia (blood flow). The recognition of these two conditions from DFU images, and the underlying dataset design, follow the framework established by Goyal et al. [12] and the DFUC 2021 Challenge [8].

4.2 Task formulation

The model is trained as a 4-class softmax over `[none, infection, ischaemia, both]` — the DFUC 2021 label structure [7,8]. From the softmax the application derives two independent binaries:

- $P(\text{infection}) = P(\text{infection}) + P(\text{both}) \rightarrow \text{threshold } 0.41 \rightarrow \text{Present / Not Present}$.
- $P(\text{ischaemia}) = P(\text{ischaemia}) + P(\text{both}) \rightarrow \text{threshold } 0.61 \rightarrow \text{Impaired / Adequate}$.

This derived-binary formulation handles the rare ischaemia class far better than a direct 4-way argmax. The two binaries feed two UI rows and a top-level risk badge: none → Normal, infection → Infection Detected, ischaemia → Impaired Blood Flow, both → High Risk.

4.3 Datasets

Datasets are pooled with one representative image per wound:

Dataset	Wounds	none / infection / ischaemia / both	Notes
DFUC 2021 (train.csv)	5,955	2552 / 2555 / 227 / 621	Whole-image labels; 3,994 unlabeled rows dropped.
Part B (DFUNet)	1,447	595 / 642 / 24 / 186	Jointly labeled (same wound in both Infection & Ischaemia folders) → maps to 4-class.
Pooled	7,402	3147 / 3197 / 251 / 807	Part B lifts the scarce ischaemia / both classes.

Table 7. Pooled infection/ischaemia dataset. Class counts show how Part B reinforces the under-represented ischaemia and both classes.

Part B's pre-made augmentation copies (`_M` mirror, `_R` rotations) are collapsed to one crop per wound: because the backbone is frozen, augments add no signal and risk leakage. Cross-validation uses `StratifiedGroupKFold(5)` grouped by wound, eliminating augment- and patch-level leakage.

4.4 Architecture

- **Encoder:** the same frozen CLIP ViT-B/32 [4] as Model 2 (`clip_backbone_fp16.tflite`), verified byte-identical (MD5) to the backbone the head was trained against.
- **Head:** L2-normalize → Dense(256, ReLU) → Dropout(0.5) → Dense(4, softmax), trained with label smoothing 0.05 and class-balanced weights.
- **Model selection:** the head was chosen through two grouped-CV bake-offs (`model3_infection_experiments*.py`). The MLP beat linear logistic regression; label smoothing then beat weight-decay, focal loss, and a 5-seed ensemble — at zero extra size (a single 262 KB head).
- **Export:** fp16 TFLite; Keras↔TFLite parity max-abs-diff of 1.2e-04.

4.5 Results (5-fold grouped CV, out-of-fold)

Metric	Infection	Ischaemia
AUC	0.890	0.987
F1 @ tuned threshold	0.829	0.872
Threshold	0.41	0.61

Table 8. Out-of-fold cross-validated performance for the two derived binaries.

Aggregate performance: macro-binary AUC 0.939, macro-F1 0.850; 4-class accuracy 0.776. Per-source AUCs are near-identical across DFUC 2021 and Part B, confirming that pooling generalises.

Progression: linear logistic regression 0.838 / 0.963 (macro-F1 0.788) → MLP 0.885 / 0.985 (0.842) → MLP + label-smoothing 0.890 / 0.987 (0.850).

4.6 Files

File	Size	Role
infection_ischaemia_head.tflite	262 KB	512-d embedding → 4-class softmax
infection_ischaemia_head_meta.json	—	classes, derived-binary map, thresholds, badge rule

Table 9. Deployed artefacts for Model 3. Note the head is under 0.3 MB.

4.7 Limitations

LIMITATIONS OF MODEL 3

Infection reflects broad clinical criteria (redness, warmth, pain, discharge), not exudate alone; neuropathic patients may under-present, so the output is not a sole diagnostic basis. The ischaemia AUC of 0.987 is near ceiling, whereas infection (0.890) retains the remaining headroom — the next levers are wound-crop preprocessing via Model 1 or additional Part B crops, both at essentially zero extra deployment size.

5 Shared Preprocessing

5.1 CLIP preprocessing (Models 2 & 3)

This pipeline must match CLIP exactly, or classification accuracy collapses:

1. Decode the image to RGB.
2. Resize so the shorter side = 224 (bicubic).
3. Centre-crop to 224×224 .
4. Divide by 255 $\rightarrow [0,1]$.
5. Per-channel normalise: mean = [0.48145466, 0.45782750, 0.40821073], std = [0.26862954, 0.26130258, 0.27577711].
6. Feed NHWC float32 [1, 224, 224, 3], channel order RGB.

5.2 Model 1 preprocessing

Model 1 uses its own pipeline: resize to 320×320 , divide by 255 $\rightarrow [0,1]$. The [-1,1] rescale required by the MobileNetV2 backbone is baked into the exported graph.

6 Application Integration

- **Assets:** `assets/models/` holds all four TFLite files (`model1_wound_fp16.tflite`, `clip_backbone_fp16.tflite`, `tissue_head.tflite`, `infection_ischaemia_head.tflite`) plus the two `*_meta.json` files.
- **Capture flow:** photo $\rightarrow$ preview $\rightarrow$ analysis. The reference-object scale calibration and manual-depth steps that formerly sat between the photo and the result have been removed, so capture now proceeds directly to analysis and measurements run uncalibrated (see §2.4).
- **Service:** `AiService` (`lib/features/wound/analysis/services/ai_service.dart`) loads all models, runs Model 1 for measurements, and runs the CLIP backbone once for both heads.
- **Result model:** `AnalysisResult` carries the measurements (length, width, area), `tissueType`, `infection`, `ischaemia`, and `riskBadge`. The legacy `depth` field is retained in the data class but is no longer populated or displayed (it defaults to 0), preserving database and synchronisation compatibility.
- **UI:** the results screen shows the length and width measurement cards, the risk badge, the tissue findings, and the Infection and Blood-Flow rows; the depth row and the “not calibrated” banner were removed with the depth/calibration steps. History and persistence (local SQLite, schema v18) store infection/ischaemia; the interface is EN/AR localised.
- **Offline verification:** `python mobile_export/reference_inference.py <image.jpg>` runs the backbone once and prints both heads' outputs.

7 References

References are grouped into methods/architectures and datasets. The dataset entries [7]–[12] are the citations mandated by the dataset providers (Manchester Metropolitan University, Diabetic Foot Ulcer Challenge) and are reproduced as officially required; they must be cited in any publication that uses the DFUC collections. Adapt the formatting to the citation style required by your institution (the entries below follow a plain numbered convention).

Methods and architectures

1. O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation,” in *Medical Image Computing and Computer-Assisted Intervention (MICCAI) 2015*, LNCS vol. 9351, Springer, 2015, pp. 234–241. arXiv:1505.04597.
2. M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted Residuals and Linear Bottlenecks,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520. arXiv:1801.04381.
3. N. Abraham and N. M. Khan, “A Novel Focal Tversky Loss Function with Improved Attention U-Net for Lesion Segmentation,” in *Proc. IEEE 16th Int. Symp. Biomedical Imaging (ISBI)*, 2019, pp. 683–687. arXiv:1810.07842.
4. A. Radford, J. W. Kim, C. Hallacy, et al., “Learning Transferable Visual Models from Natural Language Supervision” (CLIP), in *Proc. 38th Int. Conf. Machine Learning (ICML)*, PMLR vol. 139, 2021, pp. 8748–8763. arXiv:2103.00020.
5. M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning Robust Visual Features without Supervision,” 2023. arXiv:2304.07193.

Datasets

6. M. K. Dhar et al., “Wound Tissue Segmentation in Diabetic Foot Ulcer Images Using Deep Learning: A Pilot Study,” 2024. arXiv:2406.16012. (DFUTissue dataset; code and data: github.com/uwm-bigdata/DFUTissueSegNet.)
7. M. H. Yap, C. Kendrick, N. D. Reeves, M. Goyal, J. M. Pappachan, and B. Cassidy, “Development of Diabetic Foot Ulcer Datasets: An Overview,” in *Diabetic Foot Ulcers Grand Challenge*, 2021, pp. 1–18. arXiv:2201.00163. doi:10.1007/978-3-030-94907-5_1.
8. M. H. Yap, B. Cassidy, J. M. Pappachan, C. O’Shea, D. Gillespie, and N. D. Reeves, “Analysis Towards Classification of Infection and Ischaemia of Diabetic Foot Ulcers,” in *2021 IEEE EMBS International Conference on Biomedical and Health Informatics (BHI)*, IEEE, 2021, pp. 1–4.
9. B. Cassidy, N. D. Reeves, J. M. Pappachan, D. Gillespie, C. O’Shea, S. Rajbhandari, A. G. Maiya, E. Frank, A. J. Boulton, D. G. Armstrong, and B. Najafi, “The DFUC 2020 Dataset: Analysis Towards Diabetic Foot Ulcer Detection,” *touchREVIEWS in Endocrinology*, vol. 17, no. 1, p. 5, 2021.
10. M. Goyal, N. D. Reeves, S. Rajbhandari, and M. H. Yap, “Robust Methods for Real-Time Diabetic Foot Ulcer Detection and Localization on Mobile Devices,” *IEEE Journal of Biomedical and Health Informatics*, vol. 23, no. 4, pp. 1730–1741, 2018.
11. M. Goyal, N. D. Reeves, A. K. Davison, S. Rajbhandari, J. Spragg, and M. H. Yap, “DFUNet: Convolutional Neural Networks for Diabetic Foot Ulcer Classification,” *IEEE Transactions on Emerging Topics in Computational Intelligence*, vol. 4, no. 5, pp. 728–739, 2018.
12. M. Goyal, N. D. Reeves, S. Rajbhandari, N. Ahmad, C. Wang, and M. H. Yap, “Recognition of Ischaemia and Infection in Diabetic Foot Ulcers: Dataset and Techniques,” *Computers in Biology and Medicine*, vol. 117, p. 103616, 2020.

NOTE ON CITATIONS

Entries [7]–[12] are the citations required by the DFUC dataset providers and should be reproduced in full in any publication using these datasets; the overview paper [7] documents the origin and interconnection of the Challenge collections. Still to be added by the author: a citation/approval statement for the doctor-annotated `dfu-extra-labels` set, and, if reproduced from a specific implementation, a source for the CLIP preprocessing constants. The datasets are released for non-commercial research use only — confirm each licence before publication.